Pumping Lemma 1 & 2 – Complete Conceptual Notes

When students say **"Pumping Lemma 1 and 2"**, they are referring to two different pumping lemmas used for proving that certain languages do **not** belong to specific language classes.

1. Pumping Lemma for **Regular Languages** (DFAs)
2. Pumping Lemma for **Context-Free Languages** (PDAs / Grammars)

1. Pumping Lemma for Regular Languages (PL1)

Used to prove: A language is **NOT Regular** (cannot be accepted by any DFA).

Core Logic (Pigeonhole Principle)

A DFA has a fixed number of states. If a string longer than the number of states is processed, the automaton must revisit at least one state, creating a loop.

String Decomposition: s = xyz

1. **x**: Part before the loop
2. **y**: The loop itself (must be non-empty)
3. **z**: Part after the loop

Pumping Rule: $xy^iz \in L$ for all $i \geq 0$

Classic Example: $L = \{ a^nb^n \}$

Choose $s = a^pb^p$. Since the first p symbols are all 'a', the loop y contains only 'a'. Pumping creates more a's than b's, violating the language rule. Hence, L is not regular.

2. Pumping Lemma for Context-Free Languages (PL2)

Used to prove: A language is **NOT Context-Free** (cannot be accepted by any PDA).

Core Logic (Parse Trees)

Very long strings require tall parse trees. Along a long root-to-leaf path, some non-terminal must repeat, creating two pumpable sections.

String Decomposition: s = uvxyz

1. **v** and **y**: Pumped together
2. **u, x, z**: Fixed parts

Pumping Rule: $uv^ix\ y^iz \in L$ for all $i \geq 0$

Classic Example: $L = \{ a^nb^nc^n \}$

Choose $s = a^pb^pc^p$. The constraint $|vxy| \leq p$ prevents vxy from covering all three blocks. Pumping changes counts of at most two symbols, breaking equality. Hence, L is not context-free.

Summary Cheat Sheet

Feature	PL1 (Regular)	PL2 (Context-Free)
Structure	Linear DFA States	Parse Tree
Parts	x, y, z	u, v, x, y, z

What Loops?	y only	v and y
Pump Formula	xy■z	uv■xy■z
What it disproves	a■b■	a■b■c■

How to Solve Questions – The "Demon Game"

1. Demon chooses pumping length p
2. You choose a difficult string s using p
3. Demon splits the string to try to survive
4. You choose i (usually 0 or 2)
5. If the pumped string breaks the rule, you win